1 Pre-boarding Training Courses

Below is the list of the training courses to be completed before joining as an intern. This doesn't require a Udemy paid license.

Language	Course name	Udemy course link (free)
Java	Core Java Programming in 2025 - Part 1	https://www.udemy.com/course/core-java-programming-in-2025-part-1/
Java	Core Java Programming in 2025 - Part 2	https://www.udemy.com/course/core-java-programming-in-2025-part-2/
Java	Core Java Programming in 2025 - Part 3	https://www.udemy.com/course/core-java-programming-in-2025-part-3/
Python	Python for Beginners [2025]: Zero to Hero	https://www.udemy.com/course/python-for-beginners-zero-to-hero-e/
Python	Python Mastery Unleashed: Advanced Concepts	https://www.udemy.com/course/python-mastery-unleashed-advanced-concepts/
Python	Exception Handling in Python 3 [**Optional**]	https://www.udemy.com/course/exception-handling-in-python-3-try-except-else-finally-raise/
Python	Try Django 1.11 // Python Web Development [**Optional**]	https://www.udemy.com/course/try-django-v1-11-python-web-development/

1.1 Instructions for training courses

1. **Log in to Udemy using your personal email ID** and complete the Python and Java courses. These are free courses.
   Note: Two additional Python learning courses are optional.
2. **After completing all required trainings,** take a screenshot of your **'My Learning'** page in Udemy.
   - Ensure that each course shows **100% completion.**
   - Your **username** should be clearly visible in the screenshot.
3. **Save the screenshot as a JPG file and share it via email.**
   - Reply to the email you received **from OpenText** regarding this training
   - **Update the subject line to include your full name**.
4. Please ensure that you **complete all training** and **share the screenshot before joining us as an intern.**

2 Project Details – AI-Powered Cloud Cost Optimizer (LLM-Driven)

2.1 Objective

This assignment evaluates your ability to design and implement an AI-powered cost optimization system. You will work on extracting project requirements, generating realistic synthetic billing data, and producing multi-cloud optimization recommendations using LLMs. Completing this assignment will demonstrate your proficiency in backend development, LLM integration, and structured report generation.

2.2 Assignment Overview

Build a Cloud Cost Optimizer tool where users provide a plain-English project description, and the system:
1. Extracts a structured project profile.
2. Generates realistic, budget-aware synthetic cloud billing.
3. Analyzes costs against the budget.
4. Produces actionable cost optimization recommendations.
5. Operates through a menu-driven CLI.

2.3 Scope & Features

2.3.1 Mandatory Features

1. **Project Profile Extraction**
   - Input: project_description.txt (free-form text).
   - Output: project_profile.json with fields like name, budget_inr_per_month, tech_stack, and non_functional_requirements.
   - Must be generated exclusively via an LLM call (no rule-based systems).

Ex: {
  "name": "Ecommerce Market Analysis Tool",
  "budget_inr_per_month": 3000,
  "description": " Hi, I want to build a market analysis tool for e-commerce. The tool should track the highest-selling products each month. For the front end, I am using React, for the backend Node.js, and MongoDB for storing the data. I'll use Nginx as a proxy server and AWS for hosting. My monthly budget is 3000.",
  "tech_stack": {
    "frontend": "react",
    "backend": "nodejs",
    "database": "mongodb",
    "proxy": "nginx",
    "hosting": "aws"
  },
  "non_functional_requirements": []
}

2. **Synthetic Billing Generation**
- Input: project_profile.json.
- Output: mock_billing.json (12–20 realistic records).
- Records include: month, service, usage_quantity, unit, cost_inr, region, etc.
- LLM-generated, cloud-agnostic, budget-aware.
Ex: [

```
 {
  "month": "2025-01",
  "service": "EC2",
  "resource_id": "i-ecommerce-web-01",
  "region": "ap-south-1",
  "usage_type": "Linux/UNIX (on-demand)",
  "usage_quantity": 720,
  "unit": "hours",
  "cost_inr": 900,
  "desc": "Ecommerce web server"
 },
 {
  "month": "2025-01",
  "service": "EC2",
  "resource_id": "i-ecommerce-api-01",
  "region": "ap-south-1",
  "usage_type": "Linux/UNIX (on-demand)",
  "usage_quantity": 360,
  "unit": "hours",
  "cost_inr": 450,
  "desc": "Ecommerce API server"
 },
]
```


3. **Cost Analysis & Recommendations**
- Inputs: project_profile.json, mock_billing.json.
- Output: cost_optimization_report.json containing total cost, costs by service, budget variance, and 6–10 recommendations.
- Recommendations must be multi-cloud (AWS, Azure, GCP, open-source/free-tier options).
Ex: {

```
 "project_name": "Ecommerce Market Analysis Tool",
 "analysis": {
  "total_monthly_cost": 4050,
  "budget": 3000,
  "budget_variance": 1050,
  "service_costs": {
   "EC2": 1650,
   "RDS": 900,
   "S3": 350,
   "Lambda": 300,
   "CloudWatch": 100,
   "Load Balancer": 100,
   "Route 53": 50,
   "SES": 100,
   "CloudFront": 500
  },
  "high_cost_services": {
   "EC2": 1650
```



```
    },
    "is_over_budget": true
  },
  "recommendations": [
    {
      "title": "Migrate MongoDB to Open-Source MongoDB",
      "service": "MongoDB",
      "current_cost": 900,
      "potential_savings": 450,
      "recommendation_type": "open_source",
      "description": "Migrate to open-source MongoDB, reducing costs and improving flexibility.",
      "implementation_effort": "medium",
      "risk_level": "medium",
      "steps": [
        "Assess MongoDB usage and determine if open-source is suitable",
        "Migrate to open-source MongoDB, ensuring data integrity",
        "Update application code to accommodate open-source MongoDB"
      ],
      "cloud_providers": [
        "AWS",
        "Azure",
        "GCP"
      ]
    },
    {
      "title": "Utilize AWS Free Tier for EC2",
      "service": "EC2",
      "current_cost": 1650,
      "potential_savings": 825,
      "recommendation_type": "free_tier",
      "description": "Utilize the AWS Free Tier for EC2, reducing costs and improving cost
awareness.",
      "implementation_effort": "low",
      "risk_level": "low",
      "steps": [
        "Review AWS Free Tier for EC2, understanding eligible resources and limits",
        "Update resource configurations to utilize free tier, if applicable",
        "Monitor usage and adjust configurations as needed"
      ],
      "cloud_providers": [
        "AWS"
      ]
    },
    {
      "title": "Explore Alternative Cloud Providers for EC2",
      "service": "EC2",
      "current_cost": 1650,
      "potential_savings": 825,
      "recommendation_type": "alternative_provider",
      "description": "Explore alternative cloud providers, potentially reducing costs and improving
flexibility.",
      "implementation_effort": "high",
      "risk_level": "high",
      "steps": [
        "Research and evaluate alternative cloud providers (e.g., Google Cloud, DigitalOcean)",
        "Assess migration complexities and costs",
```



```
        "Migrate EC2 resources to alternative provider, ensuring data integrity"
      ],
      "cloud_providers": [
        "Google Cloud",
        "DigitalOcean"
      ]
    },
    {
      "title": "Optimize Lambda Function Configuration",
      "service": "Lambda",
      "current_cost": 300,
      "potential_savings": 100,
      "recommendation_type": "optimization",
      "description": "Optimize Lambda function configuration, reducing costs and improving
performance.",
      "implementation_effort": "low",
      "risk_level": "low",
      "steps": [
        "Review Lambda function configuration, identifying opportunities for optimization",
        "Update function configuration, utilizing optimized settings",
        "Monitor performance and adjust configurations as needed"
      ],
      "cloud_providers": [
        "AWS",
        "Azure",
        "GCP"
      ]
    },
    {
      "title": "Right-Size EC2 Instances",
      "service": "EC2",
      "current_cost": 1650,
      "potential_savings": 825,
      "recommendation_type": "right_sizing",
      "description": "Right-size EC2 instances, ensuring optimal resource utilization   and reducing
costs.",
      "implementation_effort": "medium",
      "risk_level": "medium",
      "steps": [
        "Assess EC2 instance usage patterns, identifying opportunities for right-sizing",
        "Update instance configurations, ensuring optimal resource utilization",
        "Monitor performance and adjust configurations as needed"
      ],
      "cloud_providers": [
        "AWS",
        "Azure",
        "GCP"
      ]
    },
    {
      "title": "Implement CloudFront CDN",
      "service": "CloudFront",
      "current_cost": 500,
      "potential_savings": 250,
      "recommendation_type": "cost-effective_storage",
```



```
      "description": "Implement CloudFront CDN, reducing data transfer costs and improving
content delivery.",
      "implementation_effort": "medium",
      "risk_level": "medium",
      "steps": [
        "Review CloudFront configuration, identifying opportunities for optimization",
        "Implement CloudFront CDN, utilizing optimized settings",
        "Monitor performance and adjust configurations as needed"
      ],
      "cloud_providers": [
        "AWS"
      ]
    },
    {
      "title": "Optimize Monitoring and Logging",
      "service": "CloudWatch",
      "current_cost": 100,
      "potential_savings": 50,
      "recommendation_type": "optimization",
      "description": "Optimize monitoring and logging, reducing costs and improving performance.",
      "implementation_effort": "low",
      "risk_level": "low",
      "steps": [
        "Review CloudWatch configuration, identifying opportunities for optimization",
        "Update monitoring and logging configurations, utilizing optimized settings",
        "Monitor performance and adjust configurations as needed"
      ],
      "cloud_providers": [
        "AWS",
        "Azure",
        "GCP"
      ]
    }
  ],
  "summary": {
    "total_potential_savings": 3325,
    "savings_percentage": 82.09876543209876,
    "recommendations_count": 7,
    "high_impact_recommendations": 0
  }
}
```


4. **CLI Orchestrator**
- Provide a menu-driven CLI to enter descriptions, run the full pipeline, view summaries, and export reports.

2.3.2 Optional (Bonus) Features

- Prompt retry with stricter instructions on failure.
- JSON Schema validation.
- Azure/GCP specific billing support.
- Export HTML report.

2.4 Tech Stack & Prerequisites

- Language: Python 3.10+
- LLM Access: Hugging Face Inference API (or equivalent) Note : Make sure to use those LLM's which are good in Json Generation : Ex: meta-llama/Meta-Llama-3-8B-Instruct
- Environment Handling: .env for API keys
- CLI Tool: Python console-based (Windows-friendly)
- Validation: Strict JSON-only outputs from the LLM

2.5 Tasks

1. Profile Extraction
   - User Enter the Project Description in the CLI and it should get saved in project_description.txt → generate project_profile.json.
   - Fail clearly if LLM response is invalid.
2. Billing Generation
   - Input: project_profile.json.
   - Output: mock_billing.json with 12–20 records.
   - Ensure data distribution across compute, DB, storage, networking, monitoring.

3. Cost Analysis & Recommendations
   - Input: project_profile.json, mock_billing.json.
   - Output: cost_optimization_report.json.
   - Must include recommendations with savings, risks, and multi-cloud alternatives.

   Recommendation can contain an open-source free alternative of the paid version which is being used in the Project.

4. CLI Orchestration
   - Menu options:
     1. Enter new project description.
     2. Run Complete Cost Analysis.
     3. View Recommendations.
     4. Export Report.

2.6 Submission Guidelines

Submit the following:
1. Clean, modular, well-documented source code.
2. GitHub repository URL (public or access-enabled).
3. Sample artifacts: project_description.txt, project_profile.json, mock_billing.json, cost_optimization_report.json.
4. README with setup instructions, examples, and run steps.
5. Optional: A short demo video (≤5 minutes).

2.7 Pre-Submission Checklist

- Code pushed to GitHub (public access enabled).
- All mandatory features implemented and validated.
- CLI workflow is functional and user-friendly.
- README includes setup and usage instructions.
- Example input and output files included.

2.8 AI Usage & Academic Integrity

- You may use AI tools to assist, but must:
  - Cite them in the README under 'Tools Used'.
  - Ensure you understand all submitted code.
  - Avoid copy-pasting full solutions.

2.9 Contact for Queries

For any clarification, please contact your assignment coordinator.

2.10 Appendix: Example Workflow (Reference)

2.10.1 Step 1: Project Description (Input)

We are building a food delivery app for 10,000 users per month.
Budget: ₹50,000 per month.
Tech stack: Node.js backend, PostgreSQL database, object storage for images, monitoring, and basic analytics.
Non-functional requirements: scalability, cost efficiency, uptime monitoring.

2.10.2 Step 2: Project Profile (Generated by LLM)


```
{
  "name": "Food Delivery App",
  "budget_inr_per_month": 50000,
  "description": "A scalable food delivery platform serving ~10k monthly users.",
  "tech_stack": {
    "backend": "Node.js",
    "database": "PostgreSQL",
    "storage": "Object Storage (images, media)",
    "monitoring": "Basic uptime & performance",
    "analytics": "Basic user analytics"
  },
  "non_functional_requirements": ["Scalability", "Cost efficiency", "Monitoring"]
}
```


2.10.3 Step 3: Synthetic Billing (Generated by LLM)


```
[
  {"service": "Compute", "cost_inr": 15000, "desc": "Backend server"},
  {"service": "Database", "cost_inr": 10000, "desc": "PostgreSQL DB"},
  {"service": "Storage", "cost_inr": 5000, "desc": "Image storage"},
  {"service": "Monitoring", "cost_inr": 2000, "desc": "Uptime checks"},
  {"service": "Networking", "cost_inr": 7000, "desc": "Traffic costs"}
]
```


2.10.4 Step 4: Cost Optimization Report (Generated by LLM)


```
{
  "total_cost_inr": 39000,
  "budget_inr": 50000,
  "variance_inr": -11000,
  "over_budget": false,
  "recommendations": [
```


{"title": "Switch to Reserved Instances", "current_cost": 15000, "potential_savings": 4000, "cloud_providers": ["AWS", "Azure", "GCP"]},
{"title": "Use Open-Source Monitoring", "current_cost": 2000, "potential_savings": 2000, "cloud_providers": ["Open Source"]}
]
}

2.10.5 Step 5: CLI Orchestrator (Sample Flow)

> python cost_optimizer.py

1. Enter a new project description
2. Run Complete Cost Analysis
3. View Recommendations
4. Export Report

2.11 Instructions for Project work

1. **Complete the assigned project using GitHub** for version control and code management.
2. Use **Python** to implement the project.
    a. Refer to Tech Stack & Prerequisites (2.4) for framework guidelines.
3. **Thoroughly test and validate** your project to ensure it meets all specified requirements.
4. Once the **project is complete:**
    a. Please **Reply-All to the email thread** where you previously shared the status of your Python and Java trainings.
    b. Please share the link to the GitHub repository.
5. Ensure that the project is completed and shared before your internship begins.

